SIMATS ENGINEERING

COURSE CODE: CSA1412
ASSESSMENT TOOL 2

COURSE NAME: COMPILER DESIGN

COURSE OUTCOME COVERED

CO2: *Construct top-down and bottom up parsers using CFG for parsing conflicts and error recovery for efficient syntax tree generation (BL3)*

Assessment Tool Weightage: 20%

QUESTIONS: 25

Annexure A

SHORT ANSWER TEST
Duration :60 Mins On Class Total Marks:50

Code of Conduct:

I Mathusri P-192311363 (*Name / Reg No*) *certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment*.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

Mathusri P

CO2 ASSESSMENT - 1

P. Mathurri

192311363

QI.

Given grammar: S→ ABD

(i) FIRST and FOLLOW Sets

FIRST Sets:

• FIRST (D) =

{e, f

• FIRST (A) =

II

A a Db |E

BgD | dA | E Delf

FIRST (a) U FIRST (Db) U {e}

{a} U {e, f} U{e}

= [{ a, e, f, e}]

• FIRST (B) = FIRST (gD) U FIRST (dA) U {E}

{g} U {d} u {e}

• FIRST (S) = FIRST (ABD)

= FIRST (A) - {E} U [if &E FIRST(A) then FIRST (B)-{e}] U [if EE FIRST (A) and €€ FIRST (B) then FIRST (D)]

{a, e, f} u {g,d} u {e, f}

{a, d, e, f, g}

{g, d, ε}

FOLLOW Sets:

- In A→ Db

FOLLOW (S) = {$}

In S→ ABD

-

FOLLOW (A) = FIRST (BD) - {E}

{g, d, e, f

– FOLLOW (B) = FIRST (D) - {e}

{e, f}

FOLLOW (D) = FOLLOW (S) = {$}

(ii) Predictive Parsing Table

Non-terminal

54

a

S→ ABD

A→ a

FOLLOW (D) = {b}

- In B→ gD

FOLLOW (D) = FOLLOW (B) = {e, f}

In BdA

- FOLLOW (A) = FOLLOW (B) = {e, f}

FOLLOW (A) = {g, d, e, f}

FOLLOW (B) = { e, f}

FOLLOW (D) = { b, e, f, $}

b

d

e

f

9

$

SABD

A→E

B÷dA

S→ ABD

A→ Db

S→ ABD

S-ABD

A-Db

A-ε

B→E

B→E

B-gD

D→f

A

B

D

(iii) Verify LL (1) Property

No FIRST/FIRST conflicts.

For E-productions, entries are placed only in FOLLOW Set columns.

No FIRST/FOLLOW conflicts.

.. The grammar is LL (1).

(v) Design Recommendations

• Grammar can be used directly for LL (1) parsing.

No modification is required

• Predictive parser advantages:

→ Simple and efficient.

D-e

(iv) Predictive Parsing for imput string: age $

Stack

S $

Input

age$

ABD $

a BD $

BD $

age$ age$ ge$

9DD$

ge$

D $

e $

e $

e $

$

$

Action

S→ABD

Aa

match a, pop

B→ g D

match g, pop

D→e

match e, pop

Accept

7

Uses one lookahead symbol.

→ No backtracking

Easy error detection and recovery.

Q2.

Shift Reduce Parser

Given Grammar:

ε → E+E

Input String:

3 – 33

id id / (id + id)

ε

E* E

ε

> E/E

ε

(E)

ε

→ id

(i) Stack Implementation

Shift - Reduce parser uses a stack to store symbols.

- Shift: Move next input symbol

to stack.

Reduce： Replace handle by

corresponding non-terminal.

(ii) Shift - Reduce Parsing Process

Step

O

Stack

Input

Action

$

id

2

$ id

3

$ €

4

$E*

5

$ E

id

6

$E E

$ E

Accept: Input parsed

successfully.

8

$ E/

9

$E/C

id / (id +id) $ * id / (id +id) $

*id / (id + id) $

id/ (id+id) $

/ (id + id) $

/ (id + id) $

/ (id + id) $

(id + id) $

id + id) $

Shift id

Reduce Eid

Shift * Shift id

Reduce E-id

Reduce EƐ* E

Shift / Shift C

Shift id

10
$ E/Cid
+ id) $
Reduce Eid

11
$E/CE
+id) $
Shift +

(12)
$E/CE+
id) $
Shift id

13
$E/CE+id
) $
Reduce Eid

14)
$E/CE+E
) $
Reduce E→ E+E

(15)
$E/CE

) $

Shift)

16

$E/ (E)

$

Reduce E-(E)

17

$ E/E

$

(18

$ E

$

(iii) Handles and Productions Used (iv) Parser Decisions and Conflicts

Handle

id

id

*E*E*

id

id

ε +ε

(ε) *E*/E

Production Used

P!← 3

→id

33-3

E→id

P! ← 3

3+3 +3

(3) −3

EE/E

Shift decision: Take next input symbol when more symbols are needed.

- Reduce decision: When top of stack matches RHS of a production.

Shift-Reduce conflict may occur as grammar is ambiguous.

Resolution: Use operator precedence and associativity.

Precedence: *, /> +, -

/>+,-

Associativity: Left Associative

Reduce EE/E

Accept

(v) Parsing Trace Summary

The expression

id id / (id + id)

is successfully parsed

using shift and reduce operations.

Hence, the string is accepted.

(vi) Verification: All handles are correctly reduced using the productions. The final stack contains only & and input is $. Hence, the input string is accepted.

Q3.

Recursive Descent Parser

Given Grammar:

S → (L) I a

L→ S L'

Input String: (a)

Procedure L()

{

if (nextinput == '(')

s();

{ match ('(');

L1();

L();

}

match(')'); }

Procedure L1()

(i) Recursive Procedures

Procedure S()

2

L', S L'Iε

(ii) Top-Down Parsing (No Backtracking)

S

<⇒ (4)
介

⇒ (SL')

⇒ (a L')

⇒(a)

(iii) Sequence of Function Calls

else if (nextinput == 'a')

{

match ('a');

Step Function Call Input

Action

if (nextinput == '')

0

S()

(a)

Called

else

```
{  match(',');
@
match ('(')
(a)
Matched '('
error();
}
S(); L1(); }
3
L()
a)
Called

4
S()
a)
Called

5
match ('a')
a)
Matched 'a'
else
O
Li()
)
Called
return; // E
7
```

L1() returns E

)

No ',', return

}

8

9

match (')')

Accept

)

Matched ')'

$

Parsing Successfull

(iv) Acceptance

Input String: (a)

=>

←

Parsing: S (L) → (SL') → (aL') (a)

The entire input is consumed.

Result :

Accepted

(vi) Advantages of Recursive Descent Parser

Simple to design and implement.

Easy to understand and debug.

(v) Error Detection and Recovery

Example Invalid Input (a,

Parsing will reach end of input while expecting ')'. Error Detected: Missing ')'.

Recovery: Display error message and discard

remaining input or skip to next valid symbol and continue parsing.

- Efficient
-

Efficient for small to medium sized grammars.

Each non-terminal has a corresponding procedure.

• No backtracking required for LL(1) grammars.

Easy to extend and maintain.

Thus, the input string (a) is successfully parsed by the recursive descent parser.

Q4. Operator Precedence Parser

Design an +, -, *, ( ) Operator Precedence parser for the operators +, Demonstrate the parsing of id_id id.

(i) Unambiguous Grammar

E→ E+T|E-TIT

T→

T*FIF

F→ (E) | id

(iii) Operator Precedence Table

(ii) Precedence Relations

+

VAA II v

<

C

<

id

id

C

$

)
>
+,-,*.), $
+

id, (
+,-*,), $
$ is the end marker.

*
C
<
VVAVAAV
*1+
C

id $
VVA Vvv+
>

<
IAAAVAAV
>
>

<
<

*

(iv) Parsing of id id id

Stack

<
VVVIIV
<
<

>

AI

id

$

<

VV VAIV

VVIVVV

Accept

Step

1

$

2

$ id

3

$ E

4

$ E

5

$ Eid

6
$ E-E
id *
id $

* id $

* id $

7
$ E-E E*
id $

8
$ E-E
E* id
$

9
$ E-E EE
$

10
$ E-T
$

11
$ E
$

(v) Verification: The expression id- id

id

Input id * id $

Relation

id id $

$ < id

id >

Shift id

id * id $

E >

< id

Shift id

id > *

E<*

* < id

Action

Reduce id F, FT, TE

(No handle) Shift

Reduce id F, F→T, TE

Shift*

Shift id

id > $

Reduce id F, F→T, TE

*

> $

Reduce TET

$

> $ $

Reduce E-TE

Since has higher precedence than

Accept

id is parsed successfully.

the subexpression id id is reduced first.

Q5. LALR (1) Parser

Given Grammar:

S→ A A

Input String: aabb

Aa Alb

(i) Augmented Grammar

(ii) LR (0) Item Sets

s' → *S*

SAA

A →a A

A →b

Io:

12:

SAA

14:

A → b•

SAA

Aa A A→ ·b

Is :

Aa A

S→ AA.

Ab

I3 :

A → a• A

16:

I, :

Aa A

A → a A.

S' → S.
A → • b

(ii) FIRST and FOLLOW Sets

FIRST Sets:

FIRST(S) = {a, b}

FIRST (A) = {a, b}

(iv) LALR (1) Parsing Table

FOLLOW Sets:

FOLLOW (S) = { $ }

FOLLOW (A) = { a, b, $}

ACTION
GOTO

State
a
b
$
State
S

○

S3

S4

○

1

123+

56

Accept

S3

S3

33

S4

S4

r (A→b)

r (A→b)

r (A→b)

r (SAA)

123+5

4

r (A→aA)

r (AaA)

r (A→aA)

6

(v) Parsing Process for Input String: aabb

:

Step	Stack	Input	Action

1

aabb $

Shift a

2

0a3

abb $

Shift a

3

0a3a 3

bb $

Shift b

4

0 a 3 a 3 b 4

b $

Reduce Ab

5

0 a 3 A 6

b $

Reduce Aa A

6

0 a 3 A 6

b $

Reduce AaA

7

0A2

b $

Shift b

8
0A 2b 4
$
Reduce Ab

9
OA 2A 5
$
Reduce SAA

10
O Si
$
Accept

A215

111

(vi) Verification

- The input string aabb is successfully parsed.
- `All` reductions follow the given grammar productions.

- The parser reaches Accept

state.

Hence, the string aabb is Accepted by the LALR(1) parser.